Programs

1. Program to implement FCFS scheduling algorithm without arrival times.

Aim: To implement FCFS scheduling algorithm without Arrival times given and finding average Waiting time and average Turn around time.

Algorithm:

1.Start.

2.Read the number of processes n.

3.Enter the Burst Time (BT) for each process.

4.Set Waiting Time (WT) of the first process to 0.

5.For each remaining process:

WT[i] = WT[i-1] + BT[i-1]

6.Calculate Turnaround Time (TAT) for each process:

TAT[i] = WT[i] + BT[i]

7.Calculate:

*Average Waiting Time (AWT)

*Average Turnaround Time (ATAT)

8.Display the Process ID, Burst Time, Waiting Time, and Turnaround Time.

9.Stop.

Problem:

Consider the following process and calculate average WT and average TAT .Assume that the processes arrived in the order as follows:

Process	-	Burst Time
P1	-	2
P2	-	3
P3	-	4

Program:

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    float avg_wt = 0, avg_tat = 0;
    printf("Enter Burst Time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
   for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }
   printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
              i + 1, bt[i], wt[i], tat[i]);
    }

  avg_wt /= n;
    avg_tat /= n;
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

OUTPUT:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

TESTCASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ; if ($?) { gcc FCFS.c -o FCFS } ; if ($?) { .\FCFS }
Enter number of processes: 3
Enter Burst Time for each process:
P1: 2
P2: 3
P3: 4

Process Burst Time      Waiting Time    Turnaround Time
P1      2               0               2
P2      3               2               5
P3      4               5               9

Average Waiting Time = 2.33
Average Turnaround Time = 5.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ; if ($?) { gcc FCF
Enter number of processes: 4
Enter Burst Time for each process:
P1: 3
P2: 4
P3: 2
P4: 1

Process Burst Time      Waiting Time    Turnaround Time
P1      3               0               3
P2      4               3               7
P3      2               7               9
P4      1               9               10

Average Waiting Time = 4.75
Average Turnaround Time = 7.25
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\
Enter number of processes: 3
Enter Burst Time for each process:
P1: 3
P2: 5
P3: 7

Process Burst Time      Waiting Time    Turnaround Time
P1      3               0               3
P2      5               3               8
P3      7               8               15

Average Waiting Time = 3.67
Average Turnaround Time = 8.67
```

2. Program to implement FCFS scheduling algorithm with arrival times.

Aim: To implement FCFS scheduling algorithm with Arrival times given and finding average Waiting time and average Turn around time.

Algorithm:

1.Start.

2.Read the number of processes n.

3.Enter the Arrival Time (AT) and Burst Time (BT) for each process.

4.Arrange the processes in ascending order of Arrival Time.

5.Set the Completion Time (CT) of the first process:

CT = AT + BT

6.For each remaining process:

If the CPU is idle (CT < AT), set CT = AT + BT.

Otherwise, CT = CT + BT.

7.Calculate:

TAT = CT - AT

WT = TAT - BT

8.Find:

Average Waiting Time (AWT)

Average Turnaround Time (ATAT)

9.Display Process ID, AT, BT, CT, WT, and TAT.

10.Stop

Problem: Calculate the average waiting time and average turn around time for processes given below and draw a gantt chart.

Process	AT	BT
P1	0	3
P2	2	6
P3	1	5

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>

int main() {
   int n, i, j;
   int at[20], bt[20], ct[20], tat[20], wt[20];
   float avgWT = 0, avgTAT = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("\nEnter Arrival Time and Burst Time:\n");
   for(i = 0; i < n; i++) {
      printf("Process P%d\n", i + 1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
   }

   for(i = 0; i < n - 1; i++) {
      for(j = i + 1; j < n; j++) {
         if(at[i] > at[j]) {
            int temp;
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
         }
      }
   }

   ct[0] = at[0] + bt[0];


 for(i = 1; i < n; i++) {
      if(ct[i - 1] < at[i])
```

```
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i - 1] + bt[i];
    }


    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        avgWT += wt[i];
        avgTAT += tat[i];
    }

    avgWT /= n;
    avgTAT /= n;
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avgWT);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT);

    return 0;
}
```

OUTPUT:

TEST CASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 0
Burst Time: 3
Process P2
Arrival Time: 2
Burst Time: 6
Process P3
Arrival Time: 1
Burst Time: 5

P       AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      1       5       8       7       2
P3      2       6       14      12      6

Average Waiting Time = 2.67
Average Turnaround Time = 7.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 3
Burst Time: 6
Process P2
Arrival Time: 2
Burst Time: 3
Process P3
Arrival Time: 1
Burst Time: 8

P       AT      BT      CT      TAT     WT
P1      1       8       9       8       0
P2      2       3       12      10      7
P3      3       6       18      15      9

Average Waiting Time = 5.33
Average Turnaround Time = 11.00
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3

Enter Arrival Time and Burst Time:
Process P1
Arrival Time: 0
Burst Time: 4
Process P2
Arrival Time: 1
Burst Time: 3
Process P3
Arrival Time: 2
Burst Time: 6

P       AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P2      1       3       7       6       3
P3      2       6       13      11      5

Average Waiting Time = 2.67
Average Turnaround Time = 7.00
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

3. Program to implement SJP scheduling algorithm in preemptive mode(SRTF).

Aim: : To implement SJF scheduling algorithm in preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Set the current time t = 0.

5 At each unit of time:

- Find all processes that have arrived ($AT \leq t$) and are not completed.
- Select the process with the smallest remaining burst time.
- If a new process arrives with a shorter remaining time, preempt the currently running process and execute the new one.

6. Reduce the selected process's remaining burst time by 1.
7. If the remaining burst time becomes 0:

Mark the process as completed.

Calculate:

- Completion Time (CT) = Current Time + 1
- Turnaround Time (TAT) = CT − AT
- Waiting Time (WT) = TAT − BT

Repeat Steps 5–7 until all processes are completed.

Calculate:

- Average Waiting Time = ΣWT / n
- Average Turnaround Time = ΣTAT / n

Stop.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Problem: Calculate the average waiting time and average turn around time for processes given below and draw a gantt chart.

Process	AT	BT
P1	2	3
P2	1	2
P3	2	3

Program:

```
#include <stdio.h>

    int main() {
        int n, i;
        int at[20], bt[20], rt[20];
        int ct[20], tat[20], wt[20];

        int time = 0;
        int completed = 0;
        int shortest;
        int min;

        float avgWT = 0, avgTAT = 0;

        printf("Enter number of processes: ");
        scanf("%d", &n);

        // Input Arrival Time and Burst Time
        for(i = 0; i < n; i++) {
            printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
            scanf("%d %d", &at[i], &bt[i]);

            // Remaining time initially equals Burst Time
            rt[i] = bt[i];

            ct[i] = 0;
            tat[i] = 0;
            wt[i] = 0;
        }

        // SJF Preemptive / SRTF
        while(completed < n) {

            shortest = -1;
            min = INT_MAX;

            // Find process with shortest remaining time
            for(i = 0; i < n; i++) {
                if(at[i] <= time && rt[i] > 0 && rt[i] < min) {
                    min = rt[i];
                    shortest = i;
                }
            }
```

```
        if(shortest == -1) { time++;
                continue;
            }
            rt[shortest]--;
            time++;
            if(rt[shortest] == 0) {
                completed++;
                ct[shortest] = time;
                tat[shortest] = ct[shortest] - at[shortest];
                wt[shortest] = tat[shortest] - bt[shortest];
                avgWT += wt[shortest];
                avgTAT += tat[shortest];
            }
        }
    printf("\n----------------------------------------------------\n");
        printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
        printf("----------------------------------------------------\n");

        for(i = 0; i < n; i++) {
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        }
        printf("----------------------------------------------------\n");
        printf("Average Waiting Time = %.2f\n", avgWT / n);
        printf("Average Turnaround Time = %.2f\n", avgTAT / n);
        return 0;
}
```

OUTPUT:

TEST CASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 2
3
Enter Arrival Time and Burst Time of P2: 1
2
Enter Arrival Time and Burst Time of P3: 2
3

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      2       3       6       4       1
P2      1       2       3       2       0
P3      2       3       9       7       4
-----------------------------------------------------
Average Waiting Time = 1.67
Average Turnaround Time = 4.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\" ;
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 1
3
Enter Arrival Time and Burst Time of P2: 2
5
Enter Arrival Time and Burst Time of P3: 4
7

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      1       3       4       3       0
P2      2       5       9       7       2
P3      4       7       16      12      5
-----------------------------------------------------
Average Waiting Time = 2.33
Average Turnaround Time = 7.33
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS PROGRAMS\"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 2
4
Enter Arrival Time and Burst Time of P2: 5
6
Enter Arrival Time and Burst Time of P3: 1
8

-----------------------------------------------------
Process AT      BT      CT      TAT     WT
-----------------------------------------------------
P1      2       4       6       4       0
P2      5       6       12      7       1
P3      1       8       19      18      10
-----------------------------------------------------
Average Waiting Time = 3.67
Average Turnaround Time = 9.67
PS C:\Users\harsh\OneDrive\Documents\OS PROGRAMS>
```

4. Program to implement SJP scheduling algorithm in non-preemptive mode.

Aim: To implement SJP scheduling algorithm in non-preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Set the current time t = 0 and completed processes count = 0.
5. While all processes are not completed:
   - Find all processes that have arrived ($AT \leq t$) and are not completed.
   - Select the process with the smallest Burst Time (BT).
   - If no process has arrived, increment the current time by 1.

     Otherwise:
     - Execute the selected process until it finishes (without interruption).
     - Update:
       - Completion Time (CT) = t + BT
       - Turnaround Time (TAT) = CT − AT
       - Waiting Time (WT) = TAT − BT
     - Mark the process as completed.
     - Update the current time to the completion time.
6. Repeat Step 5 until all processes are completed.
7. Calculate:
   - Average Waiting Time = ΣWT / n
   - Average Turnaround Time = ΣTAT / n
9. Display Process ID, AT, BT, CT, TAT, and WT.
10. Stop.

Problem:

Calculate Average waiting time , Average turn around time using SRTF for the following processes given below.

Process	AT	BT
P1	2	3
P2	4	5
P3	2	3

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>
int main() {
        int n, i, time = 0, completed = 0;
        int at[20], bt [20], ct[20], tat[20], wt[20];
        int visited[20] = {0};
       printf("Enter number of processes: ");
        scanf("%d", &n);

        for(i = 0; i < n; i++) {
           printf("Enter Arrival Time of P%d: ", i + 1);
           scanf("%d", &at[i]);

           printf("Enter Burst Time of P%d: ", i + 1);
           scanf("%d", &bt[i]);
        }
     while(completed < n) {
           int idx = -1;
           int minBT = 9999;

           for(i = 0; i < n; i++) {
              if(at[i] <= time && visited[i] == 0 && bt[i] < minBT) {
                 minBT = bt[i];
                 idx = i;
              }
           }
```

```
        if(idx == -1) {
            time++;


    } else {
            ct[idx] = time + bt[idx];
            time = ct[idx];
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            visited[idx] = 1;
            completed++;
        }
    }

    float avgWT = 0, avgTAT = 0;

    printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
              i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
        avgWT += wt[i];
        avgTAT += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", avgWT / n);
    printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

    return 0;}
```

OUTPUT:

TEST CASE 1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 3
Enter Arrival Time of P1: 2
Enter Burst Time of P1: 3
Enter Arrival Time of P2: 4
Enter Burst Time of P2: 5
Enter Arrival Time of P3: 2
Enter Burst Time of P3: 3

Process AT      BT      CT      WT      TAT
P1      2       3       5       0       3
P2      4       5       13      4       9
P3      2       3       8       3       6

Average Waiting Time = 2.33
Average Turnaround Time = 6.00

Process returned 0 (0x0)   execution time : 36.339 s
Press any key to continue.
```

TESTCASE 2:

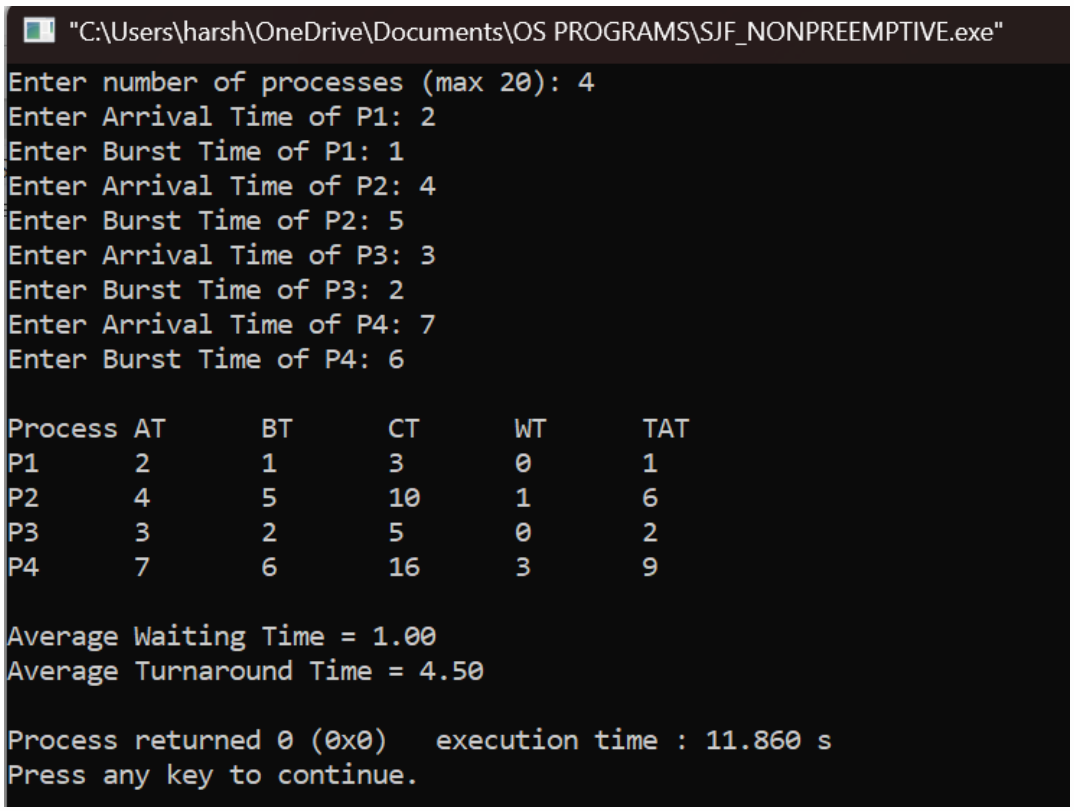
```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 3
Enter Arrival Time of P1: 1
Enter Burst Time of P1: 2
Enter Arrival Time of P2: 2
Enter Burst Time of P2: 3
Enter Arrival Time of P3: 3
Enter Burst Time of P3: 4

Process AT      BT      CT      WT      TAT
P1      1       2       3       0       2
P2      2       3       6       1       4
P3      3       4       10      3       7

Average Waiting Time = 1.33
Average Turnaround Time = 4.33

Process returned 0 (0x0)   execution time : 9.041 s
Press any key to continue.
```

TESTCASE 3:


"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\SJF_NONPREEMPTIVE.exe"
Enter number of processes (max 20): 4
Enter Arrival Time of P1: 2
Enter Burst Time of P1: 1
Enter Arrival Time of P2: 4
Enter Burst Time of P2: 5
Enter Arrival Time of P3: 3
Enter Burst Time of P3: 2
Enter Arrival Time of P4: 7
Enter Burst Time of P4: 6
Process AT BT CT WT TAT
P1 2 1 3 0 1
P2 4 5 10 1 6
P3 3 2 5 0 2
P4 7 6 16 3 9
Average Waiting Time = 1.00
Average Turnaround Time = 4.50
Process returned 0 (0x0) execution time : 11.860 s
Press any key to continue.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

1. Program to implement Priority scheduling algorithm in preemptive mode.

Aim: : To implement Priority scheduling algorithm in preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) of each process.
4. Initialize:
    - Current Time t = 0
    - Completed Processes count = 0
5. Repeat until all processes are completed:
    - Find all processes whose AT ≤ t and are not completed.
    - Select the process with the **highest priority** (smallest priority number).
    - If no process is available, increment the current time by 1.
    - Otherwise:
        - Execute the selected process completely.
        - Calculate:
            - Completion Time (CT) = Current Time + Burst Time
            - Turnaround Time (TAT) = CT − AT
            - Waiting Time (WT) = TAT − BT
        - Mark the process as completed.
        - Update the current time to the Completion Time.
6. Calculate:
    - Average Waiting Time = ΣWT / n
    - Average Turnaround Time = ΣTAT / n
7. Display Process ID, Arrival Time, Burst Time, Priority, Completion Time, Turnaround Time, and Waiting Time.
8. Stop.

Problem:

Calculate Average waiting time , Average turn around time using Priority Scheduling Algorithm in preemptive mode for the following processes given below.

Process	AT	BT	Priority
P1	1	4	6
P2	2	4	7
P3	3	6	7

Program:

```
#include <stdio.h>
 int main()
{
   int n, i, time = 0, completed = 0;
   int at[20], bt[20], rt[20], pr[20];
   int ct[20], tat[20], wt[20];
   int index, minPriority;
   float avgWT = 0, avgTAT = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time, Burst Time and Priority of P%d: ", i + 1);
      scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
      rt[i] = bt[i];
   }

   while(completed < n)
   {
      index = -1;
      minPriority = 9999;

      for(i = 0; i < n; i++)
      {
         if(at[i] <= time && rt[i] > 0)
```

```
            {

          if(pr[i] < minPriority)
                {
                   minPriority = pr[i];
                   index = i;
                }
             }
            }

            if(index == -1)
            {
               time++;
            }
            else
            {
               rt[index]--;
               time++;

               if(rt[index] == 0)
               {
                  completed++;
                  ct[index] = time;
                  tat[index] = ct[index] - at[index];
                  wt[index] = tat[index] - bt[index];
avgTAT += tat[index];
                  avgWT += wt[index];
               }
            }
         }
```

```
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    printf("-----------------------------------------------------------------\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }
    printf("-----------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

Output:

Testcase 1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 1 4 6
Enter Arrival Time, Burst Time and Priority of P2: 2 4 7
Enter Arrival Time, Burst Time and Priority of P3: 3 6 7

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      1       4       6       5       4       0
P2      2       4       7       9       7       3
P3      3       6       7       15      12      6
-----------------------------------------------------------------
Average Waiting Time = 3.00
Average Turnaround Time = 7.67

Process returned 0 (0x0)   execution time : 24.522 s
Press any key to continue.
```

Testcase 2:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 0 2 1
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 4 5 6

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      0       2       1       2       2       0
P2      2       3       4       5       3       0
P3      4       5       6       10      6       1
-----------------------------------------------------------------
Average Waiting Time = 0.33
Average Turnaround Time = 3.67

Process returned 0 (0x0)   execution time : 18.414 s
Press any key to continue.
```

Testcase 3:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority of P1: 1 2 3
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 3 4 5
Enter Arrival Time, Burst Time and Priority of P4: 4 5 6

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      1       2       3       3       2       0
P2      2       3       4       6       4       1
P3      3       4       5       10      7       3
P4      4       5       6       15      11      6
-----------------------------------------------------------------
Average Waiting Time = 2.50
Average Turnaround Time = 6.00

Process returned 0 (0x0)   execution time : 18.062 s
Press any key to continue.
```

2.Program to implement Priority scheduling algorithm in non-preemptive mode.

Aim: : To implement Priority scheduling algorithm in non-preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) of each process.
4. Set the current time t = 0 and completed processes count = 0.
5. While all processes are not completed:
    - Find all processes that have arrived ($AT \leq t$) and are not completed.
    - Select the process with the highest priority (smallest priority number).
    - If no process has arrived, increment the current time by 1.
    - Otherwise:
        - Execute the selected process until completion.
        - Calculate:
            - Completion Time (CT) = Current Time + BT
            - Turnaround Time (TAT) = CT − AT
            - Waiting Time (WT) = TAT − BT
        - Mark the process as completed.
        - Update the current time to the completion time.
6. Repeat Step 5 until all processes are completed.
7. Calculate:
    - Average Waiting Time = ΣWT / n
    - Average Turnaround Time = ΣTAT / n
8. Display Process ID, AT, BT, Priority, CT, TAT, and WT.
9. Stop.

Problem:

Calculate Average waiting time , Average turn around time using Priority Scheduling Algorithm in non-preemptive mode for the following processes given below.

Problem	AT	BT	Priority
P1	2	4	2
P2	4	6	1
P3	7	9	4
P4	5	5	3

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>
int main()
 {
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20];
    int visited[20] = {0};
    int highest, minPr;
    float avgWT = 0, avgTAT = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
       printf("Enter Arrival Time, Burst Time and Priority of P%d: ", i + 1);
       scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
    }
    while(completed < n)
    {
       highest = -1;
       minPr = 9999;
       for(i = 0; i < n; i++)
       {
          if(at[i] <= time && visited[i] == 0)
          {
             if(pr[i] < minPr)
             {
                minPr = pr[i];
                highest = i;
```

```
            }
            else if(pr[i] == minPr)
            {

                if(highest == -1 || at[i] < at[highest])
                {
                    highest = i;
                }
            }
        }
    }


    if(highest == -1)
    {
        time++;
        continue;
    }

    time += bt[highest];
    ct[highest] = time;
    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];

    avgWT += wt[highest];
    avgTAT += tat[highest];

    visited[highest] = 1;
    completed++;
}

printf("\n----------------------------------------------------------\n");
```

```
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    printf("-----------------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }

    printf("-----------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

OUTPUT:

Test Case1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority of P1: 2 4 2
Enter Arrival Time, Burst Time and Priority of P2: 4 6 1
Enter Arrival Time, Burst Time and Priority of P3: 7 9 4
Enter Arrival Time, Burst Time and Priority of P4: 5 5 3

-----------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------
P1      2       4       2       6       4       0
P2      4       6       1       12      8       2
P3      7       9       4       26      19      10
P4      5       5       3       17      12      7
-----------------------------------------------------------
Average Waiting Time = 4.75
Average Turnaround Time = 10.75

Process returned 0 (0x0)   execution time : 50.298 s
Press any key to continue.
```

Testcase 2:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 2 3 1
Enter Arrival Time, Burst Time and Priority of P2: 4 4 3
Enter Arrival Time, Burst Time and Priority of P3: 3 2 2

------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
------------------------------------------------------------
P1      2       3       1       5       3       0
P2      4       4       3       11      7       3
P3      3       2       2       7       4       2
------------------------------------------------------------
Average Waiting Time = 1.67
Average Turnaround Time = 4.67

Process returned 0 (0x0)   execution time : 20.456 s
Press any key to continue.
```

Testcase 3:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 5
Enter Arrival Time, Burst Time and Priority of P1: 1  2 3
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 3 4 5
Enter Arrival Time, Burst Time and Priority of P4: 4 5 6
Enter Arrival Time, Burst Time and Priority of P5: 5 6 7

------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
------------------------------------------------------------
P1      1       2       3       3       2       0
P2      2       3       4       6       4       1
P3      3       4       5       10      7       3
P4      4       5       6       15      11      6
P5      5       6       7       21      16      10
------------------------------------------------------------
Average Waiting Time = 4.00
Average Turnaround Time = 8.00

Process returned 0 (0x0)   execution time : 19.665 s
Press any key to continue.
```

3. Program to implement Round Robin with Quantum values 2 and 10 scheduling algorithm in non-preemptive mode.

Aim: To implement Round Robinscheduling algorithm with Quantum values 2 and 10 given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Burst Time (BT) for each process.
4. Read the Time Quantum (TQ).
5. Initialize:
    - Remaining Time (RT) = BT
    - Current Time = 0
6. Repeat until all processes are completed:
    - Traverse all processes.
    - If a process has remaining burst time:
        - If RT > TQ
            - Execute it for TQ units.
            - Reduce remaining time by TQ.
            - Increase current time by TQ.
        - Otherwise
            - Execute it for the remaining time.
            - Increase current time.
            - Calculate:
                - CT = Current Time
                - TAT = CT
                - WT = TAT − BT
7. Calculate:
    - Average Waiting Time
    - Average Turnaround Time
8. Display the results.

9. Stop.

Problem:

Process	AT	BT
P1	1	5
P2	2	6
P3	3	6

Program:

```
#include <stdio.h>

int main()
{
   int n, tq;
   int at[20], bt[20], rt[20];
   int ct[20], tat[20], wt[20];
   int completed = 0, time = 0;
   int i, found;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]);
      rt[i] = bt[i];
   }

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   while(completed < n)
   {
      found = 0;

      for(i = 0; i < n; i++)
```

```
    {
        if(at[i] <= time && rt[i] > 0)
        {
            found = 1;

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                completed++;
            }
        }
    }

    if(found == 0)
        time++;
}

float avgWT = 0, avgTAT = 0;

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
```

```
    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
  }

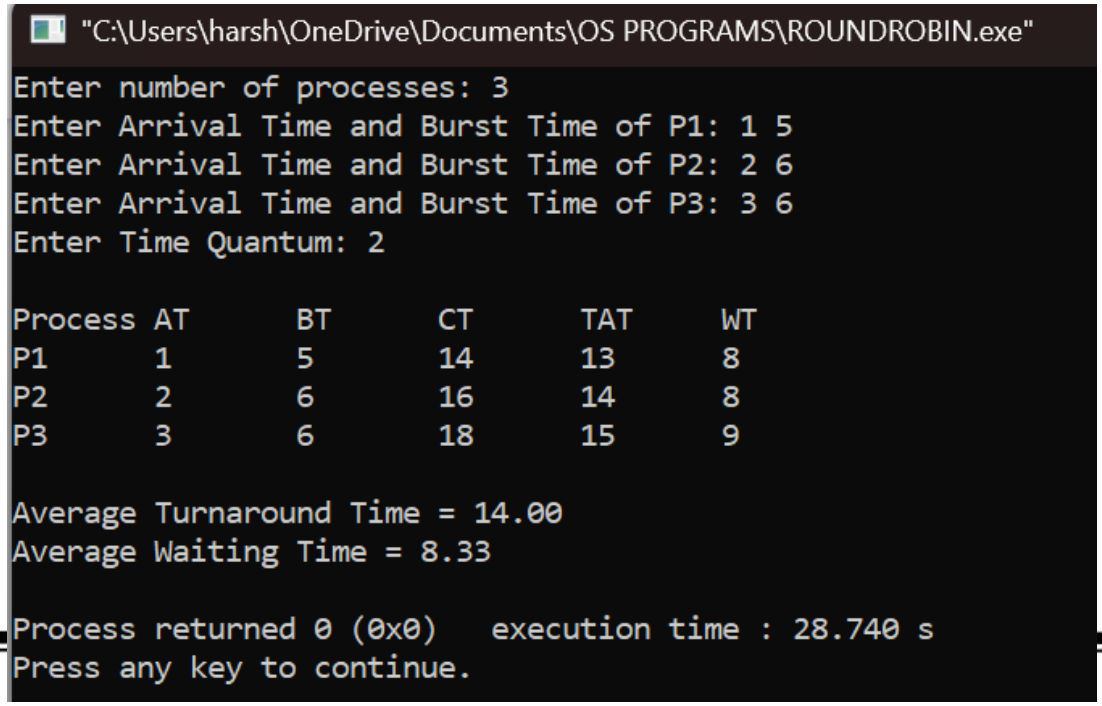
  avgWT /= n;
  avgTAT /= n;

  printf("\nAverage Turnaround Time = %.2f", avgTAT);
  printf("\nAverage Waiting Time = %.2f\n", avgWT);

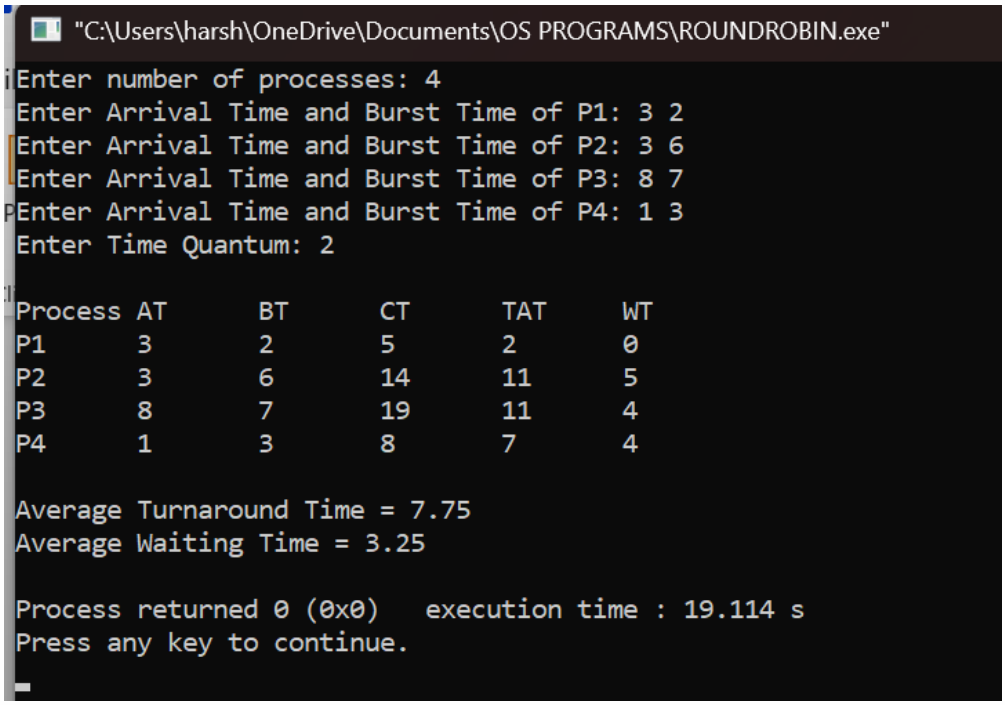
  return 0;
}
```

Output:

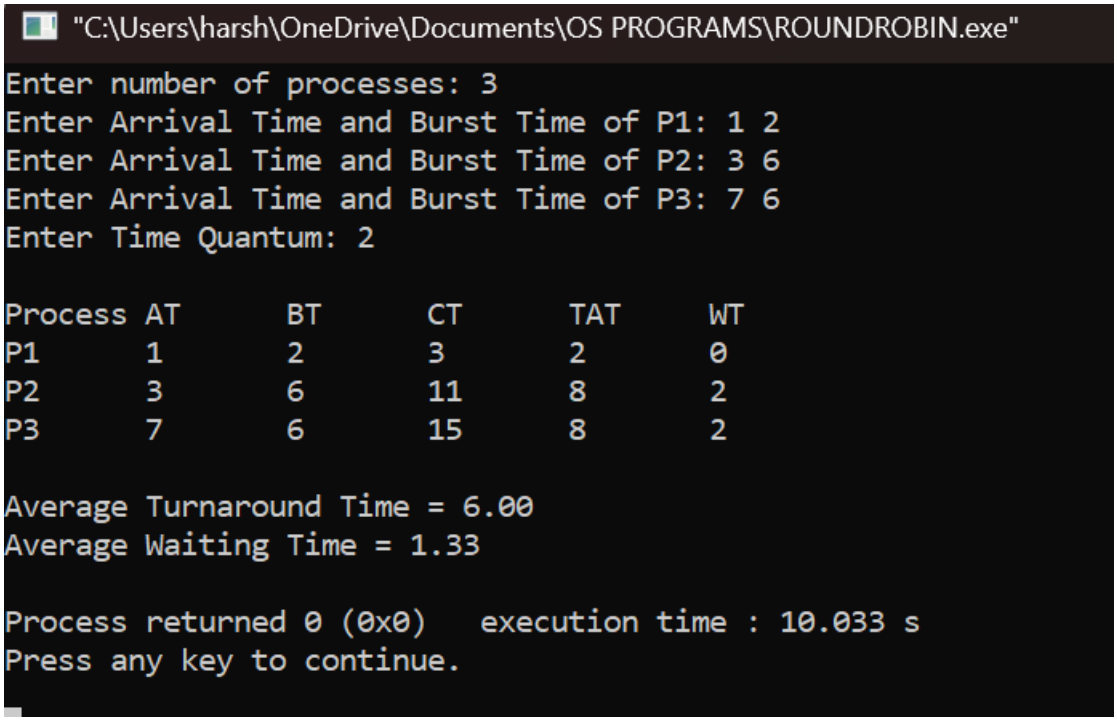
Testcase 1:



Testcase 2:



Testcase 3:


"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\ROUNDROBIN.exe"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 1 2
Enter Arrival Time and Burst Time of P2: 3 6
Enter Arrival Time and Burst Time of P3: 7 6
Enter Time Quantum: 2
Process AT BT CT TAT WT
P1 1 2 3 2 0
P2 3 6 11 8 2
P3 7 6 15 8 2
Average Turnaround Time = 6.00
Average Waiting Time = 1.33
Process returned 0 (0x0) execution time : 10.033 s
Press any key to continue.

With quantum 10:

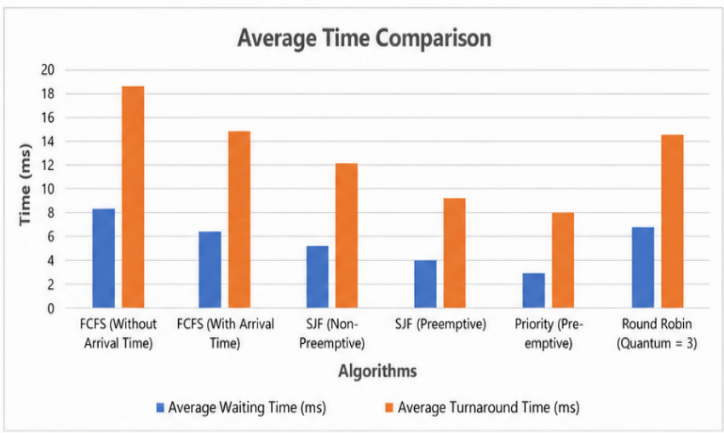
```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\ROUNDROBIN.exe"
Enter number of processes: 4
Enter Arrival Time and Burst Time of P1: 1 2
Enter Arrival Time and Burst Time of P2: 3 4
Enter Arrival Time and Burst Time of P3: 4 5
Enter Arrival Time and Burst Time of P4: 5 6
Enter Time Quantum: 10

Process AT      BT      CT      TAT     WT
P1      1       2       3       2       0
P2      3       4       7       4       0
P3      4       5       12      8       3
P4      5       6       18      13      7

Average Turnaround Time = 6.75
Average Waiting Time = 2.50

Process returned 0 (0x0)   execution time : 9.643 s
Press any key to continue.
```

Algorithm	Average Waiting Time (ms)	Average Turnaround Time (ms)
FCFS (Without Arrival Time)	8.25	18.75
FCFS (With Arrival Time)	6.25	15.25
SJF (Non-Preemptive)	5.25	11.75
SJF (Preemptive)	4.00	9.50
Priority (Pre-emptive)	3.00	8.00
Round Robin (Quantum = 3)	6.75	14.50



Name of the Laboratory:

Experiment No:_____Date:_____

WEEK-3

1 . Program to implement Banker's algorithm for a set of processes.

Aim: : To implement Banker's algorithm for a set of processes to find the safe sequence of execution of programs.

Algorithm:

1. Start.
2. Read the number of processes n and the number of resource types m.
3. Read the Allocation Matrix, which represents the resources currently allocated to each process.
4. Read the Maximum Matrix, which represents the maximum resources each process may require.
5. Read the Available Resources vector.
6. Calculate the Need Matrix using:
7. Initialize all processes as unfinished by setting Finish[i] = 0.
8. Set count = 0 to keep track of the number of processes that can safely finish.
9. Find an unfinished process Pi whose Need is less than or equal to the Available resources for every resource type.
10. If such a process is found:
    - Assume Pi completes successfully.
    - Release its allocated resources:
    - Mark the process as finished:
      Finish[i] = 1
    - Store the process in the Safe Sequence.
    - Increase count by 1.
11. Repeat Steps 9–10 until all processes are finished or no suitable process can be found.
12. If count == n:
    - The system is in a SAFE state.
    - Display the Safe Sequence.
13. Otherwise:
    - The system is in an UNSAFE state.
    - Display that no safe sequence exists.
14. Display the Need Matrix and the result.
15. Stop.

Problem:

Find the safe sequence for the following processes with Aloocation and Maximum matrix ,Available resources provided also find the Need matrixfor the processes.

Process	Allocation matrix			Max matrix			Available/Work matrix		
	A	B	C	A	B	C	A	B	C
P0	3	2	4	3	4	5	2	3	4
P1	3	4	5	3	5	8			
P2	7	8	0	1	2	3			
P3	3	2	4	4	5	2			

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Program:

```
#include <stdio.h>
 int main()
{
   int n, m;
   int alloc[10][10], max[10][10], need[10][10];
   int avail[10], finish[10] = {0}, safe[10];
   int i, j, count = 0;
   printf("============================================\n");
   printf("       BANKER'S ALGORITHM\n");
   printf("============================================\n");
   printf("Enter Number of Processes : ");
   scanf("%d", &n);
   printf("Enter Number of Resource Types : ");
   scanf("%d", &m);
   printf("\nEnter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("P%d : ", i);
      for(j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);
   }
   printf("\nEnter Maximum Matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("P%d : ", i);
      for(j = 0; j < m; j++)
         scanf("%d", &max[i][j]);
   }
   printf("\nEnter Available Resources : ");
   for(j = 0; j < m; j++)
      scanf("%d", &avail[j]);
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         need[i][j] = max[i][j] - alloc[i][j];
      }
   }
   while(count < n)
```

```
        {
            int found = 0;
            for(i = 0; i < n; i++)
            {
                if(finish[i] == 0)
                int flag = 1;
                    for(j = 0; j < m; j++)
                    {
                        if(need[i][j] > avail[j])
                        {
                            flag = 0;
                            break;
                        }
                    }
                    if(flag)
                    {
                        for(j = 0; j < m; j++)
                            avail[j] += alloc[i][j];
                        safe[count++] = i;
                        finish[i] = 1;
                        found = 1;
                    }
                }
            }
            if(found == 0)
                break;
        }
        printf("\n==========================================\n");
        printf("Need Matrix\n");
        printf("==========================================\n");
        for(i = 0; i < n; i++)
        {
            printf("P%d : ", i);
            for(j = 0; j < m; j++)
                printf("%d ", need[i][j]);
            printf("\n");
        }
        printf("\n==========================================\n");
        if(count == n)
        {
            printf("System is in SAFE State.\n");

printf("Safe Sequence : ");
```

```
        for(i = 0; i < n; i++)
        {
                printf("P%d", safe[i]);
                if(i != n - 1)
                    printf(" -> ");
            }
            printf("\n");
        }
        else
        {
            printf("System is NOT in SAFE State.\n");
        }
        printf("=============================================\n");
        return 0;
}
```

OUTPUT

TESTCASE 1:

```
=======================================
          BANKER'S ALGORITHM
=======================================
Enter Number of Processes: 4
Enter Number of Resource Types: 3

Enter Allocation Matrix:
P0: 4 5 6
P1: 7 8 9
P2: 1 2 3
P3: 7 5 3

Enter Maximum Matrix:
P0: 4 5 8
P1: 5 2 8
P2: 3 6 9
P3: 4 2 1

Enter Available Resources: 5 6 7

=======================================
             NEED MATRIX
=======================================
P0: 0 0 2
P1: -2 -6 -1
P2: 2 4 6
P3: -3 -3 -2

=======================================
System is in SAFE State.
Safe Sequence: P0 -> P1 -> P2 -> P3
=======================================
```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

TESTCASE 2:

```
==========================================
          BANKER'S ALGORITHM
==========================================
Enter Number of Processes: 3
Enter Number of Resource Types: 4

Enter Allocation Matrix:
P0: 2 3 5 6
P1: 5 7 8 9
P2: 2 6 3 5

Enter Maximum Matrix:
P0: 1 2 9 8
P1: 5 6 4 7
P2: 2 5 8 9

Enter Available Resources: 2 5 8 3

==========================================
              NEED MATRIX
==========================================
P0: -1 -1 4 2
P1: 0 -1 -4 -2
P2: 0 -1 5 4

==========================================
System is in SAFE State.
Safe Sequence: P0 -> P1 -> P2
==========================================
```

TESTCASE 3:

```
==========================================
           BANKER'S ALGORITHM
==========================================
Enter Number of Processes: 3
Enter Number of Resource Types: 3

Enter Allocation Matrix:
P0: 1 5 3
P1: 8 6 5
P2: 4 8 2

Enter Maximum Matrix:
P0: 1 5 6
P1: 7 8 9
P2: 2 4 6

Enter Available Resources: 1 5 6

==========================================
               NEED MATRIX
==========================================
P0: 0 0 3
P1: -1 2 4
P2: -2 -4 4

==========================================
System is in SAFE State.
Safe Sequence: P0 -> P1 -> P2
==========================================
```

WEEK-4

AIM:

Implementation of the following Memory Allocation Methods for fixed partition

a)First Fit b) Worst Fit c) Best Fit

ALGORITHM:

Step 1: Start

Step 2: Input

- Read the number of memory partitions n.
- Read the size of each partition into the original[] array.
- Read the number of processes m.
- Read the size of each process into the processes[] array.

Step 3: First Fit

1. Copy the original partition sizes into the first[] array.
2. Initialize all process allocations to -1.
3. For each process, search the partitions from the beginning.
4. If a partition is free and its size is greater than or equal to the process size:
   - Allocate the process to that partition.
   - Store the partition number.
   - Mark the partition as occupied by setting its size to -1.
   - Stop searching for that process.
5. If no suitable partition is found, mark the process as *Not Allocated*.
6. Display the First Fit allocation.

Step 4: Best Fit

1. Copy the original partition sizes into the best[] array.
2. Initialize all process allocations to -1.
3. For each process, search all free partitions.

4. Find the *smallest partition* whose size is greater than or equal to the process size.

5. If a suitable partition is found:

- Allocate the process to that partition.
- Store the partition number.
- Mark the partition as occupied by setting its size to -1.

6. If no suitable partition is found, mark the process as *Not Allocated*.

7. Display the Best Fit allocation.

Step 5: Worst Fit

1. Copy the original partition sizes into the worst[] array.

2. Initialize all process allocations to -1.

3. For each process, search all free partitions.

4. Find the *largest partition* whose size is greater than or equal to the process size.

5. If a suitable partition is found:

* Allocate the process to that partition.

* Store the partition number.

* Mark the partition as occupied by setting its size to -1.

6. If no suitable partition is found, mark the process as *Not Allocated*.

7. Display the Worst Fit allocation.

Step 6: Stop

PROGRAM

```
#include <stdio.h>
void firstFit(int partitions[], int n, int processes[], int m)
{
   int allocation[m];

   for (int i = 0; i < m; i++)
      allocation[i] = -1;

   for (int i = 0; i < m; i++)
   {
      for (int j = 0; j < n; j++)
      {
         if (partitions[j] >= processes[i])
         {
            allocation[i] = j;
            partitions[j] = -1;  // Partition is occupied
            break;
         }
      }
   }

   printf("\n--- First Fit ---\n");
   printf("Process\tSize\tPartition\n");

   for (int i = 0; i < m; i++)
   {
      if (allocation[i] != -1)
         printf("P%d\t%d\t%d\n", i + 1, processes[i], allocation[i] + 1);
```

```
        else
            printf ("P%d\t%d\tNot Allocated\n", i + 1, processes[i]);
    }
}

void bestFit(int partitions[], int n, int processes[], int m)
{
    int allocation[m];
        for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
    {
        int best = -1;

        for (int j = 0; j < n; j++)
        {
            if (partitions[j] >= processes[i])
            {
                if (best == -1 || partitions[j] < partitions[best])
                    best = j;
            }
        }

        if (best != -1)
        {
            allocation[i] = best;
            partitions[best] = -1;
        }
```

```
    }

    printf("\n--- Best Fit ---\n");
    printf("Process\tSize\tPartition\n");

    for (int i = 0; i < m; i++)
    {
        if (allocation[i] != -1)
            printf("P%d\t%d\t%d\n", i + 1, processes[i], allocation[i] + 1);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, processes[i]);
    }
}

void worstFit(int partitions[], int n, int processes[], int m)
{
    int allocation[m];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
    {
        int worst = -1;

        for (int j = 0; j < n; j++)
        {
            if (partitions[j] >= processes[i])
            {
```

```
            if (worst == -1 || partitions[j] > partitions[worst])
                worst = j;
        }
    }

    if (worst != -1)
    {
        allocation[i] = worst;
        partitions[worst] = -1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process\tSize\tPartition\n");

for (int i = 0; i < m; i++)
{
    if (allocation[i] != -1)
        printf("P%d\t%d\t%d\n", i + 1, processes[i], allocation[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, processes[i]);
}
}

int main()
{
    int n, m;

    printf("Enter number of partitions: ");
    scanf("%d", &n);
```

```
int original[n];

  printf("Enter size of each partition:\n");
  for (int i = 0; i < n; i++)
  {
     printf("Partition %d: ", i + 1);
     scanf("%d", &original[i]);
  }

  printf("\nEnter number of processes: ");
  scanf("%d", &m);

  int processes[m];

  printf("Enter size of each process:\n");
  for (int i = 0; i < m; i++)
  {
     printf("Process %d: ", i + 1);
     scanf("%d", &processes[i]);
  }

  // First Fit
  int first[n];
  for (int i = 0; i < n; i++)
     first[i] = original[i];

  firstFit(first, n, processes, m);
```

```
    // Best Fit
    int best[n];
    for (int i = 0; i < n; i++)
        best[i] = original[i];

    bestFit(best, n, processes, m);

    // Worst Fit
    int worst[n];
    for (int i = 0; i < n; i++)
        worst[i] = original[i];

    worstFit(worst, n, processes, m);

    return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

OUTPUT:

TESTCASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\hars
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc FITS.C -o FITS } ;
f ($?) { .\FITS }
Enter number of partitions:

                          5
Enter size of each partition:
Partition 1: 25
Partition 2: 69
Partition 3: 6
Partition 4: 78
Partition 5: 96

Enter number of processes: 3
Enter size of each process:
Process 1: 69
Process 2: 98
Process 3: 97

--- First Fit ---
Process Size    Partition
P1      69      2
P2      98      Not Allocated
P3      97      Not Allocated

--- Best Fit ---
Process Size    Partition
P1      69      2
P2      98      Not Allocated
P3      97      Not Allocated

--- Worst Fit ---
Process Size    Partition
P1      69      5
P2      98      Not Allocated
P3      97      Not Allocated
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

:

TESTCASE 2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc FITS.C -o FITS } ;
f ($?) { .\FITS }
Enter number of partitions: 5
Enter size of each partition:
Partition 1: 69
Partition 2: 98
Partition 3: 97
Partition 4: 96
Partition 5: 99

Enter number of processes: 5
Enter size of each process:
Process 1: 88
Process 2: 55
Process 3: 66
Process 4: 33
Process 5: 22

--- First Fit ---
Process Size    Partition
P1      88      2
P2      55      1
P3      66      3
P4      33      4
P5      22      5

--- Best Fit ---
Process Size    Partition
P1      88      4
P2      55      1
P3      66      3
P4      33      2
P5      22      5

--- Worst Fit ---
Process Size    Partition
P1      88      5
P2      55      2
P3      66      3
P4      33      4
P5      22      1
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc FITS.C -o FITS } ;
f ($?) { .\FITS }
Enter number of partitions: 4
Enter size of each partition:
Partition 1: 25
Partition 2: 69
Partition 3: 68
Partition 4: 59

Enter number of processes: 5
Enter size of each process:
Process 1: 56
Process 2: 36
Process 3: 23
Process 4: 31
Process 5: 21

--- First Fit ---
Process Size    Partition
P1      56      2
P2      36      3
P3      23      1
P4      31      4
P5      21      Not Allocated

--- Best Fit ---
Process Size    Partition
P1      56      4
P2      36      3
P3      23      1
P4      31      2
P5      21      Not Allocated

--- Worst Fit ---
Process Size    Partition
P1      56      2
P2      36      3
P3      23      4
P4      31      Not Allocated
P5      21      1
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

WEEK-6

AIM:

Implementation of Paging Technique of Memory Management

ALGORITHM:

Step 1: Start

Step 2: Input

- Read the size of logical memory.
- Read the size of physical memory.
- Read the size of each page.
- Calculate the number of pages.
- Calculate the number of frames.

Step 3: Create Page Table

1. Create a page table.
2. Read the frame number allocated to each page.
3. Store the page-to-frame mapping in the page table.

Step 4: Read Logical Address

1. Read the logical address.
2. Calculate the page number:
    - Page Number = Logical Address / Page Size
3. Calculate the offset:
    - Offset = Logical Address % Page Size

Step 5: Address Translation

1. Use the page number to search the page table.
2. Find the corresponding frame number.
3. Calculate the physical address:
    - Physical Address = (Frame Number × Page Size) + Offset

Step 6: Output

- Display the page number.
- Display the offset.
- Display the frame number.
- Display the physical address.

Step 7: Stop

PROGRAM:

```
#include <stdio.h>

int main()
{
    int pageSize, logicalMemory, physicalMemory;
    int numPages, numFrames;
    int pageTable[100];
    int logicalAddress;
    int pageNumber, offset;
    int frameNumber, physicalAddress;
    int i;

    printf("Enter Logical Memory Size: ");
    scanf("%d", &logicalMemory);

    printf("Enter Physical Memory Size: ");
    scanf("%d", &physicalMemory);

    printf("Enter Page Size: ");
    scanf("%d", &pageSize);
```

```
numPages = logicalMemory / pageSize;
numFrames = physicalMemory / pageSize;

printf("\nNumber of Pages = %d", numPages);
printf("\nNumber of Frames = %d\n", numFrames);

printf("\nEnter Frame Number for each Page:\n");

for (i = 0; i < numPages; i++)
{
   printf("Page %d -> Frame: ", i);
   scanf("%d", &pageTable[i]);
}

// Display Page Table
printf("\n--- Page Table ---\n");
printf("Page\tFrame\n");

for (i = 0; i < numPages; i++)
{
   printf("%d\t%d\n", i, pageTable[i]);
}

printf("\nEnter Logical Address: ");
scanf("%d", &logicalAddress);


pageNumber = logicalAddress / pageSize;
```

```
    offset = logicalAddress % pageSize;

    if (pageNumber >= numPages)
    {

     printf("\nInvalid Logical Address!\n");
        return 0;
    }
   frameNumber = pageTable[pageNumber];
  physicalAddress = (frameNumber * pageSize) + offset;
  printf("\n--- Address Translation ---\n");
  printf("Logical Address  = %d\n", logicalAddress);
  printf("Page Number      = %d\n", pageNumber);
  printf("Offset           = %d\n", offset);
    printf("Frame Number     = %d\n", frameNumber);
    printf("Physical Address = %d\n", physicalAddress);
    return 0;
}
```

OUTPUT:

TESTCASE 1:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS\"
Enter Logical Memory Size: 200
Enter Physical Memory Size: 400
Enter Page Size: 25

Number of Pages = 8
Number of Frames = 16

Enter Frame Number for each Page:
Page 0 -> Frame: 4
Page 1 -> Frame: 12
Page 2 -> Frame: 6
Page 3 -> Frame: 2
Page 4 -> Frame: 14
Page 5 -> Frame: 8
Page 6 -> Frame: 1
Page 7 -> Frame: 10

--- Page Table ---
Page    Frame
0       4
1       12
2       6
3       2
4       14
5       8
6       1
7       10

Enter Logical Address: 135

--- Address Translation ---
Logical Address = 135
Page Number = 5
Offset = 10
Frame Number = 8
Physical Address = 210
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE2:

```
cd "c:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc paging.c -o paging } ; if ($?) { .\paging }
Enter Logical Memory Size: 100
Enter Physical Memory Size: 200
Enter Page Size: 10

Number of Pages = 10
Number of Frames = 20

Enter Frame Number for each Page:
Page 0 -> Frame: 5
Page 1 -> Frame: 8
Page 2 -> Frame: 2
Page 3 -> Frame: 12
Page 4 -> Frame: 6
Page 5 -> Frame: 15
Page 6 -> Frame: 1
Page 7 -> Frame: 10
Page 8 -> Frame: 18
Page 9 -> Frame: 4

--- Page Table ---
Page    Frame
0       5
1       8
2       2
3       12
4       6
5       15
6       1
7       10
8       18
9       4

Enter Logical Address: 25

--- Address Translation ---
Logical Address = 25
Page Number = 2
Offset = 5
Frame Number = 2
Physical Address = 25
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE 3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc paging.c -o paging }
 ; if ($?) { .\paging }
Enter Logical Memory Size: 120
Enter Physical Memory Size: 240
Enter Page Size: 20

Number of Pages = 6
Number of Frames = 12

Enter Frame Number for each Page:
Page 0 -> Frame: 3
Page 1 -> Frame: 7
Page 2 -> Frame: 1
Page 3 -> Frame: 9
Page 4 -> Frame: 5
Page 5 -> Frame: 11

--- Page Table ---
Page    Frame
0       3
1       7
2       1
3       9
4       5
5       11

Enter Logical Address: 75

--- Address Translation ---
Logical Address = 75
Page Number = 3
Offset = 15
Frame Number = 9
Physical Address = 195
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

WEEK-9

1.Write a C Program to implement FCFS Disk scheduling algorithm.

AIM:To implement FCFS Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

Step 1: Start.

Step 2: Declare the variables `n`, `i`, `head`, and `totalSeek`. Initialize `totalSeek = 0`.

Step 3: Read the number of disk requests `n`.

Step 4: Read the disk requests into the array `requests[]`.

Step 5: Read the initial head position.

Step 6: Display the initial head position as the starting point of the disk request sequence.

Step 7: Repeat the following steps for each disk request from `i = 0` to `n - 1`:

\- Calculate the head movement using:

`abs(requests[i] - head)`

* Add the head movement to `totalSeek`.

* Move the disk head to the current request position.

* Display the current request in the sequence.

Step 8: Calculate the average head movement using:

\\text{Average Head Movement} = \\frac{\\text{Total Head Movement}}{n}

Step 9: Display the disk request sequence, total head movement, and average head movement.

Step 10: Stop.

PROBLEM:

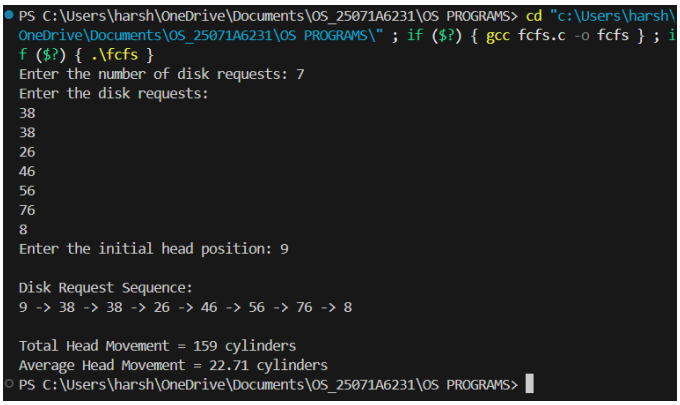
PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
   int n, i;
   int head;
   int totalSeek = 0;
   printf("Enter the number of disk requests: ");
   scanf("%d", &n);
   int requests[n];
   printf("Enter the disk requests:\n");
   for (i = 0; i < n; i++){
      scanf("%d", &requests[i]); }
   printf("Enter the initial head position: ");
   scanf("%d", &head);
   printf("\nDisk Request Sequence:\n");
   printf("%d", head);
   // FCFS: service requests in the same order entered
   for (i = 0; i < n; i++){
      totalSeek += abs(requests[i] - head);
      head = requests[i];
      printf(" -> %d", requests[i]); }
   printf("\n\nTotal Head Movement = %d cylinders", totalSeek);
   printf("\nAverage Head Movement = %.2f cylinders\n",
         (float)totalSeek / n)
```

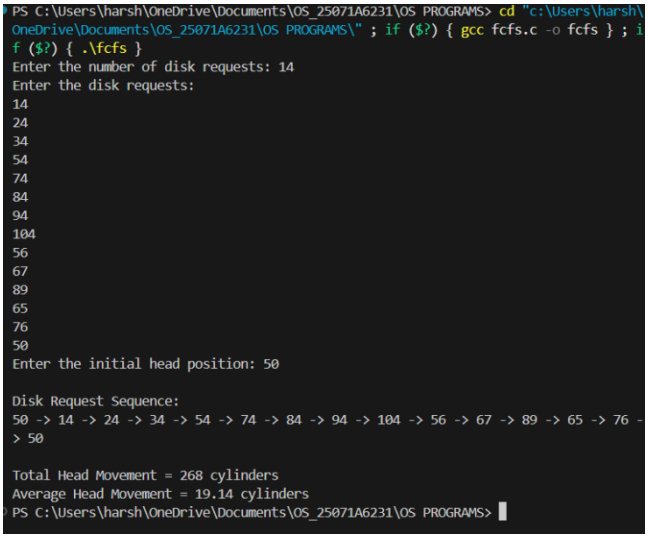
return 0;}

OUTPUT:

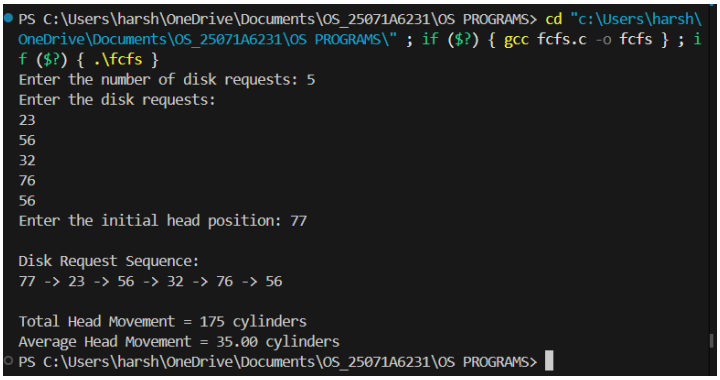
TESTCASE1:



TESTCASE2:



TESTCASE3:



2. Write a C Program to implement SSTF Disk scheduling algorithm.

AIM: To implement Shortest Seek Time First (SSTF)Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

Step 1: Start.

Step 2: Declare the variables `n`, `i`, `j`, `head`, `totalSeek`, `min`, `diff`, and `index`. Initialize `totalSeek = 0`.

Step 3: Read the number of disk requests `n`.

Step 4: Read the disk requests into the array `requests[]`.

Step 5: Initialize all elements of the `visited[]` array to `0`, indicating that no request has been serviced.

Step 6: Read the initial head position.

Step 7: Display the initial head position as the starting point of the disk request sequence.

Step 8: Repeat the following steps until all requests are serviced:

* Set `min = 9999` and `index = -1`.

* Compare the current head position with every unvisited disk request.

* Calculate the seek distance using:

`diff = abs(requests[j] - head)`

4. Find the unvisited request with the minimum seek distance.

5. Add the minimum seek distance to `totalSeek`.

6. Move the head to the selected request.

7. Mark the selected request as visited by setting `visited[index] = 1`.

8. Display the selected request in the disk request sequence.

Step 9: Calculate the average head movement.

Step 10: Display the total head movement and average head movement.

Step 11: Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int n, i, j;
    int head, totalSeek = 0;
    int min, diff, index;
    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
    int requests[n];
    int visited[n];
    for (i = 0; i < n; i++) {
        visited[i] = 0;  }
    printf("Enter the disk requests:\n");
    for (i = 0; i < n; i++)  {
        scanf("%d", &requests[i]); }
    printf("Enter the initial head position: ");
    scanf("%d", &head);
    printf("\nDisk Request Sequence:\n");
    printf("%d", head);
    for (i = 0; i < n; i++) {
        min = 9999;
        index = -1;
        for (j = 0; j < n; j++) {
            if (visited[j] == 0)   {
                diff = abs(requests[j] - head);
```

```
            if (diff < min)   {
                min = diff;
                index = j; }
}
}
    totalSeek += min;
    head = requests[index];
    visited[index] = 1;
    printf(" -> %d", head);}
  printf("\n\nTotal Head Movement = %d cylinders",   totalSeek);
  printf("\nAverage Head Movement = %.2f cylinders\n",  (float)totalSeek/n);
  return 0;
}
```

OUTPUT:

TESTCASE 1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc sstf.c -o sstf } ; i
f ($?) { .\sstf }
Enter the number of disk requests: 6
Enter the disk requests:
45
64
34
25
64
65
Enter the initial head position: 55

Disk Request Sequence:
55 -> 64 -> 64 -> 65 -> 45 -> 34 -> 25

Total Head Movement = 50 cylinders
Average Head Movement = 8.33 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

TESTCASE2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc sstf.c -o sstf } ; i
f ($?) { .\sstf }
Enter the number of disk requests: 6
Enter the disk requests:
24
86
7
85
4
3
Enter the initial head position: 100

Disk Request Sequence:
100 -> 86 -> 85 -> 24 -> 7 -> 4 -> 3

Total Head Movement = 97 cylinders
Average Head Movement = 16.17 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc sstf.c -o sstf } ; i
f ($?) { .\sstf }
Enter the number of disk requests: 4
Enter the disk requests:
23
54
23
67
Enter the initial head position: 80

Disk Request Sequence:
80 -> 67 -> 54 -> 23 -> 23

Total Head Movement = 57 cylinders
Average Head Movement = 14.25 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

3.Write a C Prpgram to implement SCAN Disk Scheduling Algorithm.

AIM: To implement SCAN Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

1.Start.

2.Read the number of requests, disk size, request queue, initial head position, and direction.

3.Sort the disk requests in ascending order.

4.Display the initial head position.

5.If the direction is right:

Service all requests greater than or equal to the current head position in ascending order.

Move the head to the end of the disk.

Reverse the direction.

Service the remaining requests in descending order.

6.If the direction is left:

Service all requests less than or equal to the current head position in descending order.

Move the head to the beginning of the disk.

Reverse the direction.

Service the remaining requests in ascending order.

7.Calculate the total head movement.

8.Calculate the average head movement by dividing the total by the number of requests.

9.Display the results.

10.Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n){
    int i, j, temp;
    for (i = 0; i < n - 1; i++)   {
        for (j = 0; j < n - i - 1; j++)      {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
               a[j + 1] = temp;   }}}}
int main()
{
    int n, diskSize, head, direction;
    int i, j, totalSeek = 0;
    int requests[100];
    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk size: ");
    scanf("%d", &diskSize);
    printf("Enter the disk requests:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &requests[i]);    }
  printf("Enter the initial head position: ");
    scanf("%d", &head);
    printf("Enter direction (1 for right, 0 for left): ");
    scanf("%d", &direction);
```

```
sort(requests, n);
    printf("\nDisk Request Sequence:\n");
    printf("%d", head);
    if (direction == 1)  {
       for (i = 0; i < n; i++)   {
          if (requests[i] >= head) {
             totalSeek += abs(requests[i] - head);
             head = requests[i];
             printf(" -> %d", head);}}
       if (head != diskSize – 1) {
          totalSeek += abs((diskSize - 1) - head);
          head = diskSize - 1;
          printf(" -> %d", head);}
       for (i = n - 1; i >= 0; i--)   {
          if (requests[i] < head) {
             totalSeek += abs(requests[i] - head);
             head = requests[i];
             printf(" -> %d", head);}}
    else  {
       for (i = n - 1; i >= 0; i--){
        if (requests[i] <= head){
             totalSeek += abs(requests[i] - head);
             head = requests[i];
             printf(" -> %d", head);  }}
       if (head != 0){
          totalSeek += head
```

```
    head = 0;
            printf(" -> %d", head);}
        for (i = 0; i < n; i++){
            if (requests[i] > head){
                totalSeek += abs(requests[i] - head);
                head = requests[i];
                printf(" -> %d", head);}}}
    printf("\n\nTotal Head Movement = %d cylinders",
            totalSeek);
    printf("\nAverage Head Movement = %.2f cylinders\n",
            (float)totalSeek / n);
    return 0;
}
```

OUTPUT:

TESTCASE1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc scan.c -o scan } ; i
f ($?) { .\scan }
Enter the number of disk requests: 6
Enter the disk requests:
12
26
56
57
36
29
Enter the initial head position: 50
Enter direction (1 for right, 0 for left): 1

Disk Request Sequence:
50 -> 56 -> 57 -> 56 -> 36 -> 29 -> 26 -> 12

Total Head Movement = 52 cylinders
Average Head Movement = 8.67 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TEST CASE2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc scan.c -o scan } ; i
f ($?) { .\scan }
Enter the number of disk requests: 5
Enter the disk requests:
45
65
58
26
25
Enter the initial head position: 25
Enter direction (1 for right, 0 for left): 1

Disk Request Sequence:
25 -> 25 -> 26 -> 45 -> 58 -> 65 -> 58 -> 45 -> 26 -> 25

Total Head Movement = 80 cylinders
Average Head Movement = 16.00 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc scan.c -o scan } ; i
f ($?) { .\scan }
Enter the number of disk requests: 5
Enter the disk requests:
54
63
89
65
23
Enter the initial head position: 25
Enter direction (1 for right, 0 for left): 1

Disk Request Sequence:
25 -> 54 -> 63 -> 65 -> 89 -> 65 -> 63 -> 54 -> 23

Total Head Movement = 130 cylinders
Average Head Movement = 26.00 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

WEEK-10

1.Write a C Program to implement C-SCAN Disk scheduling algorithm.

AIM: To implement C-SCAN Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

1.Start.

2.2Read the number of requests, disk size, request queue, and initial head position.

3.Sort the disk requests in ascending order.

4.Display the initial head position.

5.Service all requests greater than or equal to the current head position in ascending order.

6.Move the head to the end of the disk.

7.Move the head from the end to the beginning of the disk without servicing requests during the return.

8.Service the remaining requests in ascending order.

9.Calculate the total head movement.

10.Calculate the average head movement.

11.Display the disk request sequence, total head movement, and average head movement.

12.Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n){
    int i, j, temp;
    for (i = 0; i < n - 1; i++){
        for (j = 0; j < n - i - 1; j++){
            if (a[j] > a[j + 1]){
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;}}}}
int main(){
    int n, diskSize, head;
    int i, totalSeek = 0;
    int requests[100];
    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk size: ");
    scanf("%d", &diskSize);
    printf("Enter the disk requests:\n");
    for (i = 0; i < n; i++){
        scanf("%d", &requests[i])}
    printf("Enter the initial head position: ");
    scanf("%d", &head);
    sort(requests, n);
```

```
    printf("\nDisk Request Sequence:\n");
    printf("%d", head);
    for (i = 0; i < n; i++){
        if (requests[i] >= head){
            totalSeek += abs(requests[i] - head);
            head = requests[i];
            printf(" -> %d", head);}}
    if (head != diskSize – 1){
        totalSeek += abs((diskSize - 1) - head);
        head = diskSize - 1;
        printf(" -> %d", head);}
    totalSeek += diskSize - 1;
    head = 0;
    printf(" -> %d", head);
    for (i = 0; i < n; i++){
        if (requests[i] < head){
            totalSeek += abs(requests[i] - head);
            head = requests[i];
            printf(" -> %d", head);}}
    printf("\n\nTotal Head Movement = %d cylinders",    totalSeek);
    printf("\nAverage Head Movement = %.2f cylinders\n", (float)totalSeek / n);
    return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

OUTPUT:

TESTCASE1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc C-scan.c -o C-scan }
 ; if ($?) { .\C-scan }
Enter the number of disk requests: 6
Enter the disk requests:
14
04
62
54
41
34
Enter the initial head position: 50

Disk Request Sequence:
50 -> 54 -> 62 -> 0 -> 4 -> 14 -> 34 -> 41

Total Head Movement = 115 cylinders
Average Head Movement = 19.17 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc C-scan.c -o C-scan }
 ; if ($?) { .\C-scan }
Enter the number of disk requests: 4
Enter the disk requests:
34
56
78
90
Enter the initial head position: 56

Disk Request Sequence:
56 -> 56 -> 78 -> 90 -> 0 -> 34

Total Head Movement = 158 cylinders
Average Head Movement = 39.50 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc C-scan.c -o C-scan }
 ; if ($?) { .\C-scan }
Enter the number of disk requests: 7
Enter the disk requests:
12
34
54
43
65
765
56
Enter the initial head position:
8

Disk Request Sequence:
8 -> 12 -> 34 -> 43 -> 54 -> 56 -> 65 -> 765 -> 0

Total Head Movement = 1522 cylinders
Average Head Movement = 217.43 cylinders
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

2. Write a C Program to implement LOOK Disk scheduling algorithm.

AIM: To implement LOOK Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

1.Start.

2.Read the number of disk requests, request queue, initial head position, and disk direction.

3.Sort the disk requests in ascending order.

4.Find the position of the initial head in the sorted request list.

5.If the direction is right:

Service all requests greater than or equal to the head in ascending order.

Then service the remaining requests in descending order.

6.If the direction is left:

Service all requestsess than or equal to the head in descending order.

Then service the remaining requests in ascending order.

7.Calculate the seek time for each movement.

8.Calculate the total and average seek time.

9.Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n){
    int i, j, temp;
    for (i = 0; i < n - 1; i++){
        for (j = 0; j < n - i - 1; j++){
            if (a[j] > a[j + 1])
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;}}}}
int main(){
    int n, i, head;
    int totalSeek = 0;
    int current;
    int requests[100];
    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk requests:\n");
    for (i = 0; i < n; i++){
        scanf("%d", &requests[i]);}
    printf("Enter the initial head position: ");
    scanf("%d", &head);
    sort(requests, n);
    current = head;
    printf("\nLOOK Order: %d", current);
```

```
    for (i = 0; i < n; i++){
    if (requests[i] >= head){
            totalSeek += abs(current - requests[i]);
            current = requests[i];
            printf(" -> %d", current);}}
    for (i = n - 1; i >= 0; i--){
        if (requests[i] < head){
            totalSeek += abs(current - requests[i]);
            current = requests[i];
            printf(" -> %d", current);}
    printf("\n\nTotal Seek Time = %d", totalSeek);
    printf("\nAverage Seek Time = %.2f\n", (float)totalSeek / n);
    return 0;
}
```

OUTPUT:

TESTCASE1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc look.c -o look } ; i
f ($?) { .\look }
Enter the number of disk requests: 9
Enter the disk requests:
56
34
67
36
27
74
5
88
75
Enter the initial head position: 70

LOOK Order: 70 -> 74 -> 75 -> 88 -> 67 -> 56 -> 36 -> 34 -> 27 -> 5

Total Seek Time = 101
Average Seek Time = 11.22
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc look.c -o look } ;
f ($?) { .\look }
Enter the number of disk requests: 7
Enter the disk requests:
34
87
54
34
76
32
59
Enter the initial head position: 78

LOOK Order: 78 -> 87 -> 76 -> 59 -> 54 -> 34 -> 34 -> 32

Total Seek Time = 64
Average Seek Time = 9.14
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc look.c -o look } ; i
f ($?) { .\look }
Enter the number of disk requests: 6
Enter the disk requests:
89
25
65
06
36
26
Enter the initial head position: 76

LOOK Order: 76 -> 89 -> 65 -> 36 -> 26 -> 25 -> 6

Total Seek Time = 96
Average Seek Time = 16.00
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

3.Write a C Prpgram to implement C-LOOK Disk Scheduling Algorithm.

AIM: To implement C-LOOK Disk Scheduling Algorithm and find Total head movement and Average Head movement.

ALGORITHM:

1.Start.

2.Read the number of disk requests, request queue, and initial head position.

3.Sort the disk requests in ascending order.

4.Find the requests greater than or equal to the initial head.

5.Service these requests in ascending order.

6.After reaching the highest request, jump to the smallest request.

7.Service the remaining requests in ascending order.

8.Calculate the seek time for each movement, including the circular jump.

9.Calculate the total and average seek time.

10.Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n){
    int i, j, temp;
    for (i = 0; i < n - 1; i++){
        for (j = 0; j < n - i - 1; j++){
            if (a[j] > a[j + 1]){
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;}}}}
int main(){
    int n, i, head;
    int totalSeek = 0;
    int current;
    int requests[100];
    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk requests:\n");
    for (i = 0; i < n; i++){
        scanf("%d", &requests[i]);}
    printf("Enter the initial head position: ");
    scanf("%d", &head);
    sort(requests, n);
    current = head;
    printf("\nC-LOOK Order: %d", current);
```

```
    for (i = 0; i < n; i++){
        if (requests[i] >= head){
            totalSeek += abs(current - requests[i]);
            current = requests[i];
            printf(" -> %d", current);}}
    for (i = 0; i < n; i++) {
        if (requests[i] < head) {
            totalSeek += abs(current - requests[i]);
            current = requests[i];
            printf(" -> %d", current);}}
    printf("\n\nTotal Seek Time = %d", totalSeek);
    printf("\nAverage Seek Time = %.2f\n",
          (float)totalSeek / n);
    return 0;
}
```

OUTPUT:

TESTCASE1:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc c-look.c -o c-look }
 ; if ($?) { .\c-look }
Enter the number of disk requests: 6
Enter the disk requests:
22
3
33
44
55
66
Enter the initial head position: 14

C-LOOK Order: 14 -> 22 -> 33 -> 44 -> 55 -> 66 -> 3

Total Seek Time = 115
Average Seek Time = 19.17
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE2:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc c-look.c -o c-look }
 ; if ($?) { .\c-look }
Enter the number of disk requests: 6
Enter the disk requests:
99
88
77
66
55
44
Enter the initial head position: 50

C-LOOK Order: 50 -> 55 -> 66 -> 77 -> 88 -> 99 -> 44

Total Seek Time = 104
Average Seek Time = 17.33
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

TESTCASE3:

```
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS> cd "c:\Users\harsh\
OneDrive\Documents\OS_25071A6231\OS PROGRAMS\" ; if ($?) { gcc c-look.c -o c-look }
 ; if ($?) { .\c-look }
Enter the number of disk requests: 5
Enter the disk requests:
78
34
65
78
54
Enter the initial head position: 12

C-LOOK Order: 12 -> 34 -> 54 -> 65 -> 78 -> 78

Total Seek Time = 66
Average Seek Time = 13.20
PS C:\Users\harsh\OneDrive\Documents\OS_25071A6231\OS PROGRAMS>
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: File Allocation Strategies

Experiment No: ____ Date: _____

Aim: Simulate all file allocation strategies. A. Sequential, B. Indexed, C. Linked

A. Sequential

Algorithm:

1. **Start**.
2. Input disk block status, total number of blocks, starting block, and file length.
3. Check whether all blocks from start to start + length - 1 are free.
4. If all blocks are free, allocate them to the file.
5. Display the allocation status and allocated block numbers.
6. **Stop**.

B. Indexed

Algorithm:

1. **Start**.
2. Input index block number and number of file blocks n.
3. Check whether the index block is free.
4. Input the required block numbers and check whether all blocks are free.
5. If all blocks are free, allocate the index block and all required blocks.
6. Display the index block and its allocated block mapping.
7. **Stop**.

C. Linked

Algorithm:

1. **Start**.
2. Input starting block number and number of blocks n.
3. Input the block numbers forming the linked chain.
4. Check whether all required blocks are free.
5. Allocate the blocks and link them in sequence, ending with -1.
6. Display the linked block sequence.
7. **Stop**.

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

_____

Experiment No: ____ **Date:** _____

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

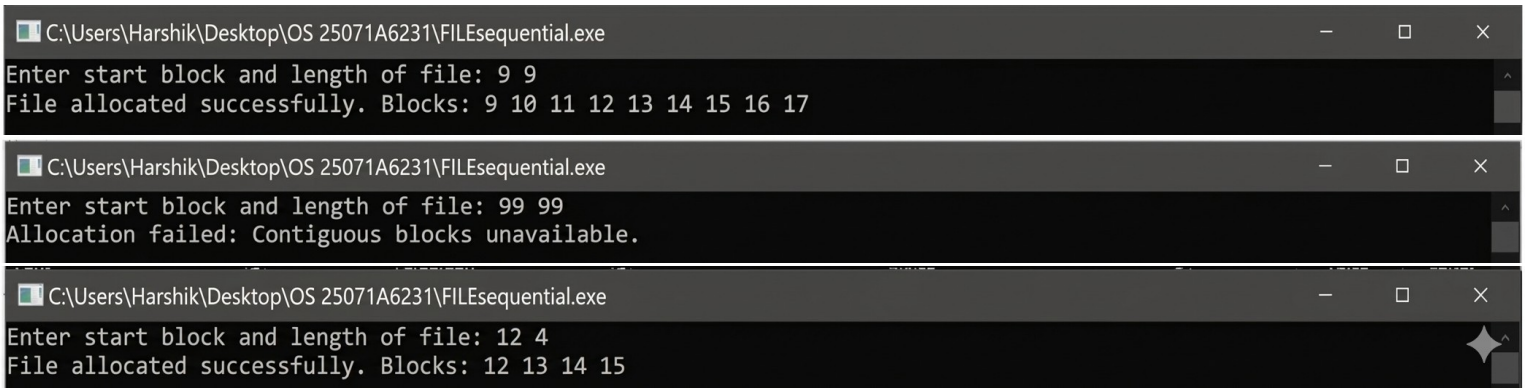
A. Sequential File allocation

Program:

```
#include <stdio.h>

int main() {
    int disk[50] = {0}, start, len, alloc = 1;
    printf("Enter start block and length of file: ");
    scanf("%d %d", &start, &len);
    for (int i = start; i < start + len; i++) if (disk[i]) alloc = 0;
    if (alloc) {
        printf("File allocated successfully. Blocks: ");
        for (int i = start; i < start + len; i++) {
            disk[i] = 1; printf("%d ", i);
        }
        printf("\n");
    } else printf("Allocation failed: Contiguous blocks unavailable.\n");
    return 0;
}
```

Testcases:



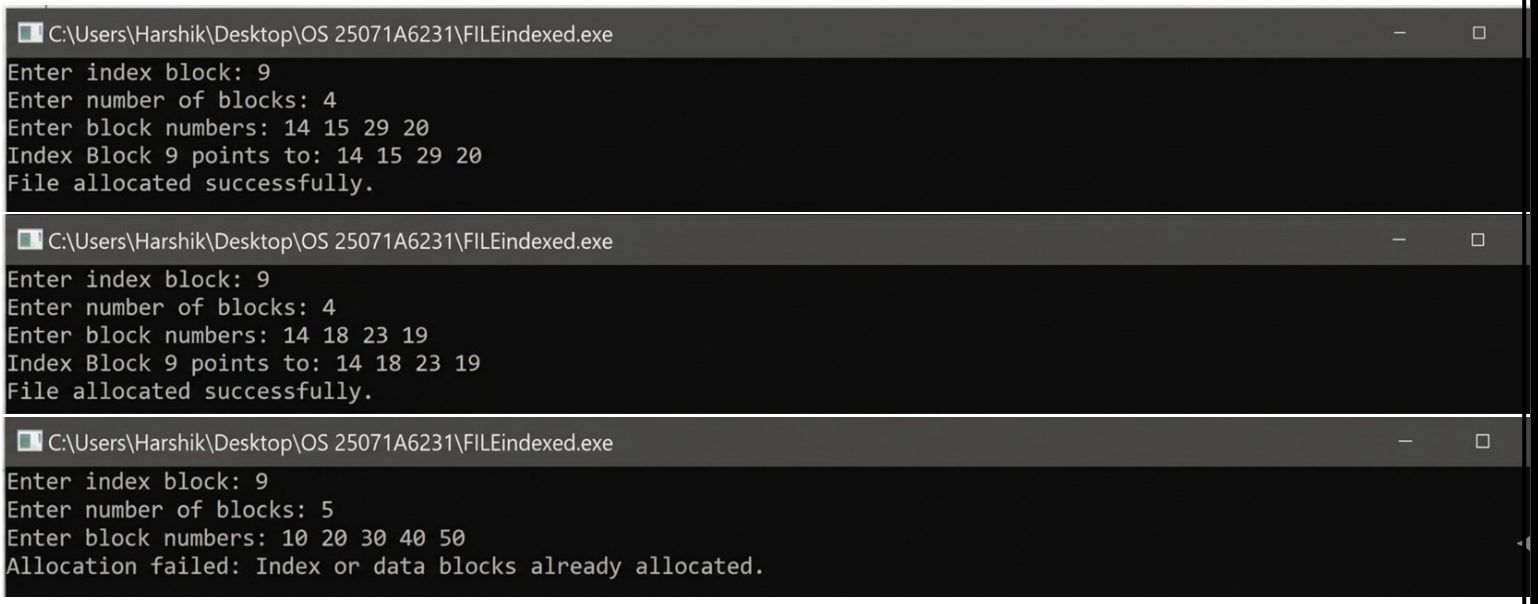
B. Indexed File Allocation

Program:

```
#include <stdio.h>

int main() {
   int disk[50] = {0}, idx, n, b[20], alloc = 1;
   printf("Enter index block: "); scanf("%d", &idx);
   printf("Enter number of blocks: “); scanf("%d", &n);
   printf(“Enter block numbers: ");
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   if (disk[idx]) alloc = 0;
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      disk[idx] = 1; printf("Index Block %d points to: ", idx);
      for (int i = 0; i < n; i++) { disk[b[i]] = 1; printf("%d ", b[i]); }
      printf("\nFile allocated successfully.\n");
   } else printf("Allocation failed: Index or data blocks already allocated.\n");
   return 0;
}
```

Testcases:



C. Linked

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6231\n");
   int disk[50] = {0}, n, b[20], alloc = 1;
   printf("Enter number of blocks:  "); scanf("%d", &n);
   printf("Enter block sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      printf("Linked Allocation Sequence: ");
      for (int i = 0; i < n; i++) {
         disk[b[i]] = 1; printf("%d -> ", b[i]);
      }
      printf("NULL\nFile allocated successfully.\n");
   } else printf("Allocation failed: One or more blocks already allocated.\n");
   return 0;
}
```

Testcases:

